

Desenvolvimento de um Assembler para a Arquitetura RISC-V32I

Demétrius Mendes Miranda Rocha
232012974

Lucas Silva Nóbrega
180035096

João Gabriel Melo de Lima
241032617

Marcus Vinicius Balbino Cavalcante
160135974

Samuel Carvalho Dos Santos
211010388

Resumo—Este artigo apresenta o desenvolvimento de um assembler para um subconjunto da arquitetura RISC-V32I. A aplicação foi implementada em Python, em arquivo único, com o objetivo de processar arquivos Assembly com extensão `.asm`, interpretar instruções e dados, resolver rótulos e gerar arquivos de inicialização de memória no formato MIF. O trabalho aborda os fundamentos da codificação de instruções, a organização das seções `.text` e `.data`, a metodologia adotada na implementação e os resultados obtidos com a geração dos arquivos `.text.mif` e `.data.mif`. Dessa forma, o projeto relaciona conceitos de Organização e Arquitetura de Computadores, linguagem Assembly, representação hexadecimal, manipulação de arquivos e funcionamento básico de um assembler.

Palavras-chave—RISC-V32I, Assembly, assembler, Python, MIF, arquitetura de computadores.

Tabela I
INFORMAÇÕES GERAIS DO LABORATÓRIO

Disciplina	Organização e Arquitetura de Computadores
Turma	03
Professor	Flávio Vidal
Semestre	1º/2026
Laboratório	Laboratório 01
Tema	Assembler para Assembly RISC V32I

I. INTRODUÇÃO

A linguagem Assembly é uma representação simbólica de baixo nível utilizada para descrever instruções executadas diretamente por um processador. Por permitir uma relação mais direta com registradores, memória e fluxo de execução, seu estudo é fundamental em Organização e Arquitetura de Computadores.

Na arquitetura RISC-V, as instruções são codificadas em palavras de 32 bits e seguem formatos padronizados, como R, I, S, B, U e J. A variante RV32I define um conjunto básico de instruções inteiras, incluindo operações aritméticas, lógicas, acessos à memória, desvios condicionais e saltos. Cada formato determina a posição dos campos necessários para a codificação, como `opcode`, `rd`, `rs1`, `rs2`, `funct3`, `funct7` e imediato.

O processo de montagem, realizado por um assembler, consiste em converter instruções escritas em Assembly para

código de máquina. Para isso, é necessário identificar mnemônicos, interpretar operandos, reconhecer registradores, tratar valores imediatos, processar rótulos e organizar as seções do arquivo, principalmente `.text`, referente às instruções, e `.data`, referente aos dados.

Neste projeto, foi desenvolvido um assembler em Python para um subconjunto da arquitetura RISC-V32I. A aplicação recebe um arquivo Assembly com extensão `.asm`, processa as instruções e os dados declarados e gera arquivos de saída no formato `.mif`, utilizado para inicialização de memórias em ambientes de simulação e projetos digitais.

Dessa forma, o projeto aplica conceitos como codificação de instruções, organização de memória, representação hexadecimal, manipulação de arquivos e funcionamento básico de um assembler, permitindo compreender as etapas necessárias para transformar um programa Assembly em uma representação compatível com o hardware ou com ambientes de simulação.

II. OBJETIVOS

O objetivo principal deste laboratório é desenvolver uma aplicação capaz de atuar como assembler para um subconjunto de instruções da arquitetura RISC V32I. A ferramenta deve receber um arquivo de entrada em Assembly, processar as seções de texto e dados, traduzir as instruções para hexadecimal e gerar arquivos de saída no formato MIF.

De forma específica, o projeto busca compreender a estrutura da arquitetura RISC V32I, interpretar instruções Assembly, reconhecer registradores, imediatos e rótulos, montar os campos binários das instruções suportadas, gerar a representação hexadecimal correspondente e produzir arquivos de inicialização de memória para as regiões de instruções e dados.

III. FUNDAMENTAÇÃO TEÓRICA

A. Arquitetura RISC-V32I

A arquitetura RISC-V é baseada no princípio RISC, caracterizado por um conjunto de instruções reduzido, regular e organizado para simplificar a implementação do processador. Neste projeto, utiliza-se a variante RV32I, que corresponde ao conjunto base inteiro de 32 bits. Nessa arquitetura, as instruções possuem tamanho fixo de 32 bits e operam sobre 32 registradores inteiros de propósito geral, identificados de `x0` a `x31` [1].

O registrador $\times 0$ possui comportamento especial, pois mantém sempre o valor zero, descartando qualquer tentativa de escrita. Os demais registradores podem armazenar operandos, resultados intermediários, endereços e valores utilizados durante a execução do programa. Além da identificação numérica, os registradores também possuem nomes definidos pela ABI, como `zero`, `ra`, `sp`, `t0`, `s0` e `a0`, que facilitam a leitura do código Assembly. Durante a montagem, esses nomes são convertidos para seus respectivos códigos de registrador.

As instruções da arquitetura RISC-V32I são organizadas em formatos padronizados, como R, I, S, B, U e J. Cada formato define a posição dos campos dentro da palavra de 32 bits, incluindo `opcode`, `rd`, `rs1`, `rs2`, `funct3`, `funct7` e campos de imediato. O formato R é usado em operações entre registradores, como `add` e `sub`; o formato I em instruções com imediatos, cargas e saltos indiretos; o formato S em armazenamentos na memória; o formato B em desvios condicionais; o formato U em imediatos superiores; e o formato J em saltos incondicionais.

B. Processo de Montagem

O assembler é responsável por converter instruções escritas em Assembly para código de máquina. Para isso, o programa deve ler o arquivo de entrada, remover comentários e espaços desnecessários, identificar diretivas, rótulos, instruções e dados, além de interpretar registradores, imediatos e operandos.

Quando há rótulos, o assembler também precisa associá-los aos seus respectivos endereços. Essa etapa é importante porque instruções de desvio e salto dependem do cálculo de deslocamentos relativos entre a instrução atual e o destino indicado pelo rótulo. Ao final do processo, os campos de cada instrução são posicionados conforme o formato correspondente da arquitetura RISC-V32I.

C. Seções `.text` e `.data`

Em programas Assembly, as seções `.text` e `.data` organizam o conteúdo do arquivo. A seção `.text` armazena as instruções que serão executadas pelo processador, enquanto a seção `.data` contém dados estáticos, como constantes, vetores e valores inicializados.

Essa separação é importante porque instruções e dados costumam ocupar regiões distintas da memória. No assembler desenvolvido, o conteúdo da seção `.text` é convertido para a memória de instruções, enquanto os valores da seção `.data` são direcionados para a memória de dados.

D. Formato MIF

O formato MIF, ou *Memory Initialization File*, é utilizado para representar o conteúdo inicial de uma memória em ferramentas de simulação e projetos digitais [3]. Esse arquivo permite definir previamente os valores armazenados em determinados endereços de memória.

No contexto deste projeto, são gerados arquivos MIF separados para a memória de instruções e para a memória de dados. A memória de instruções recebe os valores hexadecimais correspondentes às instruções codificadas, enquanto a memória de dados recebe os valores declarados na seção `.data`.

```
WIDTH=32;
DEPTH=256;

ADDRESS_RADIX=HEX;
DATA_RADIX=HEX;

CONTENT BEGIN
    00 : 00000013;
    01 : 00100093;
END;
```

Listagem 1. Estrutura geral de um arquivo MIF.

IV. MATERIAIS E MÉTODOS

A. Materiais Utilizados

O projeto foi desenvolvido em linguagem Python 3.7 ou superior, em um único arquivo-fonte denominado `main.py`. Foram utilizadas apenas bibliotecas da própria linguagem, como `pathlib`, para manipulação de caminhos, `re`, para reconhecimento de padrões textuais, e `sys`, para leitura de argumentos pela linha de comando.

Como entrada, a aplicação utiliza arquivos Assembly RISC-V32I com extensão `.asm`. Como saída, são gerados arquivos no formato `.mif`, utilizados para inicialização de memória. O simulador RARS foi utilizado como referência para a sintaxe das instruções e para o mapa de memória adotado [2], considerando os endereços-base `TEXT_BASE = 0x00400000` e `DATA_BASE = 0x10010000`.

B. Procedimento Metodológico

A metodologia adotada consistiu em dividir o processo de montagem em etapas sequenciais. Inicialmente, o arquivo Assembly de entrada é validado, verificando-se a existência do arquivo e a extensão `.asm`. Em seguida, o conteúdo é lido e submetido a um pré-processamento, no qual são removidos comentários, espaços desnecessários e linhas vazias, preservando caracteres especiais quando presentes dentro de strings.

Após o tratamento inicial, o programa separa o conteúdo nas seções `.text` e `.data`. A seção `.text` é destinada à montagem das instruções, enquanto a seção `.data` é utilizada para a geração da memória de dados. Para representar o mapa de memória utilizado, foram adotados os endereços-base `TEXT_BASE = 0x00400000` e `DATA_BASE = 0x10010000`.

A montagem da seção `.text` foi realizada em duas passagens. Na primeira, são identificados os rótulos e seus respectivos endereços. Na segunda, as instruções são codificadas, utilizando as informações previamente armazenadas para calcular deslocamentos em instruções de desvio e salto. Já a seção `.data` é processada separadamente, interpretando diretivas como `.word`, `.half`, `.byte`, `.string`, `.ascii`, `.space`, `.zero` e `.align`.

C. Validação da Saída

Ao final do processo, a aplicação gera dois arquivos no formato MIF: um correspondente à memória de instruções, com sufixo `.text.mif`, e outro correspondente à memória de dados, com sufixo `.data.mif`. Cada arquivo contém

cabeçalho com profundidade, largura da palavra, base dos endereços e base dos dados, utilizando representação hexa-decimal.

Além da geração dos arquivos, o programa realiza verificações para identificar erros durante a montagem, como arquivo inexistente, extensão inválida, registrador incorreto, imediato mal formado, operando de memória inválido, rótulo não definido, opcode desconhecido, quantidade inadequada de operandos e diretiva inválida na seção `.data`. Essas verificações auxiliam na correção do arquivo de entrada e tornam o processo de montagem mais seguro.

V. IMPLEMENTAÇÃO DO ASSEMBLER

Esta seção apresenta a estrutura interna da aplicação desenvolvida para realizar a montagem de programas escritos em Assembly RISC-V32I. A implementação foi concentrada em um único arquivo Python, denominado `main.py`, conforme exigido na especificação do laboratório. O programa foi organizado em etapas responsáveis pela leitura da entrada, tratamento das linhas, separação das seções, montagem das instruções e geração dos arquivos de saída no formato MIF.

A. Estrutura Interna do Código

A implementação do assembler foi organizada em funções e classes responsáveis por etapas específicas da montagem. As funções iniciais realizam a validação do arquivo de entrada, a leitura do conteúdo, o tratamento das linhas e a separação das seções `.text` e `.data`.

Em seguida, o programa utiliza tabelas internas para reconhecer registradores e instruções suportadas. A tabela `REG_NAMES` associa os nomes numéricos e os nomes da ABI aos códigos dos registradores, enquanto a tabela `TABELA_INSTRUcoes` armazena, para cada mnemônico, o formato da instrução, o opcode, o `funct3` e, quando necessário, o `funct7`.

A partir dessas estruturas, a implementação realiza a montagem das instruções e dos dados, gerando ao final os arquivos `.text.mif` e `.data.mif`. Dessa forma, a seção de implementação passa a enfatizar a organização interna do código, enquanto a seção de materiais e métodos fica responsável por descrever o procedimento adotado no projeto.

B. Tabelas de Registradores e Instruções

Para reconhecer os registradores da arquitetura RISC-V32I, foi criada a tabela `REG_NAMES`. Essa tabela associa os nomes numéricos dos registradores, como `x0`, `x1` e `x2`, aos nomes convencionais da ABI, como `zero`, `ra`, `sp`, `t0`, `s0` e `a0`. Durante a montagem, cada registrador é convertido para seu respectivo número inteiro entre 0 e 31.

As instruções suportadas foram organizadas na tabela `TABELA_INSTRUcoes`. Para cada mnemônico, a tabela armazena o formato da instrução, o opcode, o `funct3` e, quando necessário, o `funct7`. Essa organização centraliza as informações de codificação e facilita a seleção da rotina adequada para cada tipo de instrução.

A tabela contempla instruções dos formatos R, I, S, B, U e J, incluindo `add`, `sub`, `and`, `or`, `xor`, `sll`, `srl`, `slt`, `addi`, `andi`, `ori`, `xori`, `slti`, `lw`, `lhu`, `jalr`, `sw`, `beq`, `bne`, `lui`, `auipc` e `jal`.

C. Codificação das Instruções

A codificação das instruções foi implementada por meio de classes específicas para os formatos `TipoR`, `TipoI`, `TipoS`, `TipoB`, `TipoU` e `TipoJ`. Cada classe recebe os campos necessários da instrução e os organiza na ordem definida pela arquitetura RISC-V32I, formando uma palavra binária de 32 bits.

A função principal dessa etapa é `codificar_instrucao`. Ela recebe o mnemônico, os operandos, o contador de programa atual e o dicionário de rótulos. A partir do mnemônico, o programa consulta a tabela de instruções, identifica o formato correspondente, valida a quantidade de operandos e converte registradores e imediatos.

Nos formatos B e J, utilizados em desvios e saltos, o programa calcula o deslocamento relativo entre a instrução atual e o rótulo de destino. Esse deslocamento é então reorganizado conforme a posição dos bits exigida pelo formato da instrução. Após a montagem dos campos, a instrução é convertida para inteiro e posteriormente escrita em hexadecimal no arquivo MIF.

D. Montagem das Seções `.text` e `.data`

A montagem da seção `.text` foi realizada em duas passagens. Na primeira passagem, o programa percorre as linhas da seção de instruções e identifica os rótulos, associando cada um ao endereço correspondente. O contador de programa é iniciado em `TEXT_BASE = 0x00400000` e incrementado em 4 bytes a cada instrução, pois as instruções RV32I possuem tamanho fixo de 32 bits.

Na segunda passagem, as instruções são efetivamente codificadas. Para cada linha, o mnemônico é separado dos operandos, os registradores e imediatos são interpretados e a função de codificação é chamada. Quando uma instrução utiliza um rótulo como destino, o endereço previamente armazenado é usado para calcular o deslocamento correto.

A seção `.data` é processada separadamente. Nessa etapa, o programa identifica rótulos e associa cada um a um endereço calculado a partir de `DATA_BASE = 0x10010000`. Os dados são armazenados inicialmente em um `bytearray`, permitindo a inserção de valores byte a byte e o tratamento de diretivas de diferentes tamanhos.

Foram implementadas diretivas como `.word`, `.half`, `.byte`, `.string`, `.ascii`, `.space`, `.zero` e `.align`. Ao final do processamento, os bytes são completados até múltiplos de quatro e convertidos em palavras de 32 bits no formato *little-endian*.

E. Geração dos Arquivos MIF

Após a montagem das seções, o programa gera dois arquivos de saída no formato MIF. O primeiro arquivo, com sufixo `.text.mif`, armazena as instruções codificadas. O segundo,

com sufixo `.data.mif`, armazena os dados declarados na seção `.data`.

A função responsável pela geração do arquivo escreve o cabeçalho com a profundidade da memória, a largura da palavra, a base de endereços e a base dos dados. Em seguida, cada palavra é gravada em hexadecimal, associada a um endereço sequencial.

```

1 conteudo_original = ler_arquivo(caminho_asm)
2 conteudo_limpo = tratar_ws(conteudo_original)
3
4 linhas_text, linhas_data = particionar_secoes(
5     conteudo_limpo)
6
7 palavras_data, simbolos_data = montar_data(
8     linhas_data)
9
10 simbolos_text = primeira_passagem(linhas_text)
11 dicionario_labels = {**simbolos_data, **
12     simbolos_text}
13 palavras_text = segunda_passagem(linhas_text,
14     dicionario_labels)
15
16 gerar_arquivo_mif(saida_text, palavras_text, depth
17     =16384)
18 gerar_arquivo_mif(saida_data, palavras_data, depth
19     =32768)

```

Listagem 2. Fluxo principal simplificado da aplicação.

F. Tratamento de Erros

A implementação possui verificações para identificar problemas durante a montagem. Entre os erros tratados estão arquivo inexistente, extensão inválida, registrador inválido, imediato mal formado, operando de memória incorreto, rótulo não definido, opcode desconhecido, quantidade incorreta de operandos e diretiva inválida na seção `.data`.

Quando um erro é encontrado, o programa interrompe a execução e exibe uma mensagem indicando a causa do problema. Nos casos associados a uma linha específica do arquivo Assembly, a mensagem também informa o número da linha, facilitando a correção pelo usuário.

Portanto, a implementação segue a estrutura básica de um assembler de duas passagens, com tratamento separado para dados e instruções. Essa abordagem permite reconhecer rótulos, calcular deslocamentos relativos, converter instruções para palavras de 32 bits e gerar arquivos compatíveis com o formato MIF.

VI. RESULTADOS E DISCUSSÃO

A. Resultados Obtidos

Como resultado da implementação, foi desenvolvido um assembler em Python capaz de receber um arquivo Assembly com extensão `.asm`, processar separadamente as seções `.text` e `.data` e gerar arquivos de saída no formato MIF. A aplicação utiliza interface por terminal, recebendo o caminho do arquivo de entrada pela linha de comando ou, na ausência desse argumento, utilizando um arquivo padrão.

A seção `.text` é responsável pela montagem das instruções do programa. Para isso, o assembler identifica rótulos, interpreta mnemônicos, operandos, registradores e imediatos,

e converte cada instrução para uma palavra de 32 bits. Ao final, essas palavras são gravadas no arquivo `.text.mif`.

A seção `.data`, por sua vez, é processada para armazenar os dados declarados no programa Assembly. Foram tratadas diretivas como `.word`, `.half`, `.byte`, `.string`, `.ascii`, `.space`, `.zero` e `.align`. Os dados são organizados em bytes, ajustados quando necessário por alinhamento e convertidos para palavras de 32 bits em formato *little-endian*, sendo gravados no arquivo `.data.mif`.

Tabela II
RESUMO DOS PRINCIPAIS RESULTADOS DA IMPLEMENTAÇÃO

Aspecto analisado	Resultado obtido
Linguagem utilizada	Python 3.7 ou superior
Arquivo-fonte	Arquivo único <code>main.py</code>
Entrada do programa	Arquivo Assembly com extensão <code>.asm</code>
Saídas geradas	Arquivos <code>.text.mif</code> e <code>.data.mif</code>
Registradores reconhecidos	32 registradores inteiros e nomes ABI
Formatos suportados	R, I, S, B, U e J
Seções tratadas	<code>.text</code> e <code>.data</code>
Arquivos MIF gerados	Um para instruções e outro para dados

B. Instruções e Diretivas Suportadas

A implementação contemplou as principais instruções solicitadas no laboratório, incluindo operações aritméticas, lógicas, acessos à memória, desvios condicionais e saltos. As instruções foram organizadas em uma tabela interna contendo o formato, o opcode, o `funct3` e, quando necessário, o `funct7`.

Entre as instruções implementadas estão `add`, `sub`, `and`, `or`, `xor`, `sll`, `srl`, `slt`, `addi`, `andi`, `ori`, `xori`, `slti`, `lw`, `lhu`, `sw`, `beq`, `bne`, `jal`, `jalr`, `lui` e `auipc`. Além disso, a implementação também inclui instruções de deslocamento imediato, como `slli`, `srli` e `srai`.

Tabela III
SÍNTESE QUANTITATIVA DAS FUNCIONALIDADES IMPLEMENTADAS

Métrica	Valor observado
Registradores inteiros reconhecidos	32
Formatos de instrução tratados	6
Mnemônicos presentes na tabela interna	25
Diretivas principais da seção <code>.data</code>	8
Arquivos de saída gerados	2
Passagens realizadas na seção <code>.text</code>	2

Esses resultados indicam que o assembler desenvolvido atende ao objetivo central do laboratório, pois realiza a conversão das instruções Assembly para representação hexadecimal em arquivos MIF e também trata dados declarados na seção `.data`.

C. Avaliação do Funcionamento

O funcionamento do assembler foi organizado em etapas bem definidas. Primeiro, o arquivo de entrada é validado e lido. Em seguida, comentários e espaços desnecessários são removidos, preservando caracteres especiais dentro de strings.

Depois disso, o conteúdo é separado entre as seções `.text` e `.data`.

Na montagem da seção `.text`, o uso de duas passagens foi um resultado importante da implementação. A primeira passagem permite identificar os rótulos e associá-los aos seus endereços. A segunda passagem realiza a codificação das instruções, já com os endereços dos rótulos disponíveis. Essa abordagem permite tratar corretamente instruções de desvio e salto que referenciam rótulos definidos antes ou depois da instrução atual.

Na seção `.data`, o uso de um `bytearray` permitiu armazenar dados de diferentes tamanhos antes da conversão final para palavras de 32 bits. Isso facilitou o tratamento de diretivas como `.byte`, `.half`, `.word` e strings, além de permitir a aplicação de alinhamento quando necessário.

D. Discussão dos Resultados

Os resultados obtidos mostram que a implementação conseguiu reproduzir as etapas fundamentais de um assembler. O programa realiza a leitura do código Assembly, interpreta registradores e imediatos, reconhece rótulos, codifica instruções em palavras de 32 bits e gera arquivos compatíveis com o formato MIF.

Um ponto positivo da implementação é a separação clara entre as etapas do processo de montagem. A existência de funções específicas para leitura, tratamento das linhas, particionamento das seções, montagem de dados, identificação de rótulos, codificação de instruções e geração dos arquivos MIF torna o código mais organizado e facilita futuras correções ou expansões.

Outro aspecto relevante é o tratamento de erros. A aplicação identifica problemas como arquivo inexistente, extensão inválida, registrador incorreto, imediato mal formado, operando de memória inválido, opcode desconhecido, quantidade inadequada de operandos e rótulos não definidos. Isso evita que entradas inválidas sejam processadas sem aviso e facilita a localização de falhas no arquivo Assembly.

Em relação ao desempenho, a implementação apresenta comportamento adequado para o escopo do laboratório. A seção `.data` é percorrida uma vez, enquanto a seção `.text` é percorrida duas vezes. Dessa forma, o tempo de processamento cresce de maneira aproximadamente linear em relação à quantidade de linhas do arquivo de entrada, o que é apropriado para arquivos Assembly de pequeno e médio porte.

Como pontos de melhoria, seria possível ampliar a quantidade de testes automatizados, comparar os arquivos MIF gerados com saídas de referência e padronizar melhor as mensagens exibidas ao usuário. Também seria possível aperfeiçoar a geração dos arquivos MIF para preencher de forma mais completa os endereços não utilizados ou utilizar uma representação mais compacta para regiões de memória preenchidas com zero.

VII. CONCLUSÃO

O desenvolvimento do assembler permitiu aplicar, de forma prática, conceitos fundamentais da arquitetura RISC-V32I,

como codificação de instruções, uso de registradores, organização de memória, tratamento de rótulos e geração de arquivos de inicialização no formato MIF. A implementação em Python possibilitou transformar arquivos Assembly em representações hexadecimais compatíveis com memórias de instruções e de dados.

De modo geral, o projeto atingiu seu objetivo principal ao realizar a leitura de arquivos `.asm`, processar as seções `.text` e `.data`, codificar instruções dos principais formatos da arquitetura e gerar arquivos `.mif` separados. Além disso, o uso de duas passagens na seção de instruções mostrou-se essencial para resolver rótulos e calcular corretamente deslocamentos em desvios e saltos.

Apesar dos resultados satisfatórios, a implementação ainda pode ser aprimorada com a inclusão de mais testes automatizados, melhorias nas mensagens de erro e comparação direta com saídas de referência. Ainda assim, o trabalho cumpriu sua finalidade ao demonstrar o funcionamento básico de um assembler e reforçar a compreensão da relação entre código Assembly, código de máquina e organização da memória.

REFERÊNCIAS

- [1] RISC-V International, *The RISC-V Instruction Set Manual*. [Online]. Available: <https://riscv.org/technical/specifications/>
- [2] The Third One, *RARS: RISC-V Assembler and Runtime Simulator*. [Online]. Available: <https://github.com/TheThirdOne/rars>
- [3] Intel, *Memory Initialization File (.mif) Definition*. [Online]. Available: https://www.intel.com/content/www/us/en/programmable/quartushelp/17.0/reference/glossary/def_mif.htm